Task 5: Capture and Analyze Network Traffic Using Wireshark

Objective: Capture live network packets, filter traffic streams, and identify protocol types	**Environment:** Kali Linux (VM) | Network Interface: eth0 (10.0.2.15)
Assigned Tool: Wireshark Network Analyzer v4.6.x	**Deliverables:** Task Execution Report (.docx) & Saved Capture File (.pcap)

1. Task Objective & Execution Methodology

The objective of this task is to capture live packet traffic flowing through a virtualized Kali Linux host, utilize command-line utilities to stimulate standard client-server communication, and perform multi-layer protocol inspection in Wireshark.

- **Step 1 (Interface Selection & Monitoring):** Launched Wireshark under Kali Linux. Inspected the interface traffic sparklines on the home screen. The network interface eth0 displayed steady packet activity and was selected to start live packet sniffing in promiscuous mode.

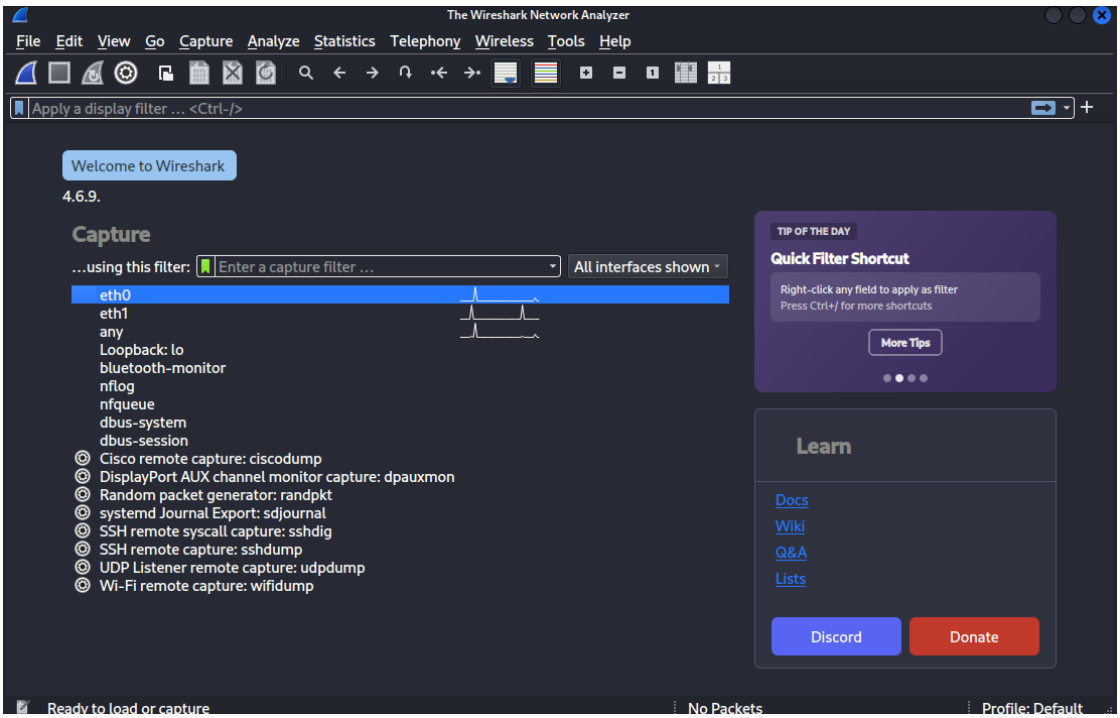


Figure 1: *Wireshark interface monitor showing live traffic activity pulses on eth0 (10.0.2.15)*

- **Step 2 (Network Traffic Generation):** Opened the Kali Linux bash terminal to trigger authentic multi-protocol traffic across the network stack:
- • **Terminal Test 1 (DNS & HTTP):** Executed 'curl -I http://neverssl.com' followed by 'curl -I http://example.com' to trigger domain name resolution and standard HTTP web requests over port 80.
- • **Terminal Test 2 (ICMP Reachability):** Executed 'ping -c 4 8.8.8.8' to generate 4 sequential Internet Control Message Protocol Echo requests to Google's public DNS server.

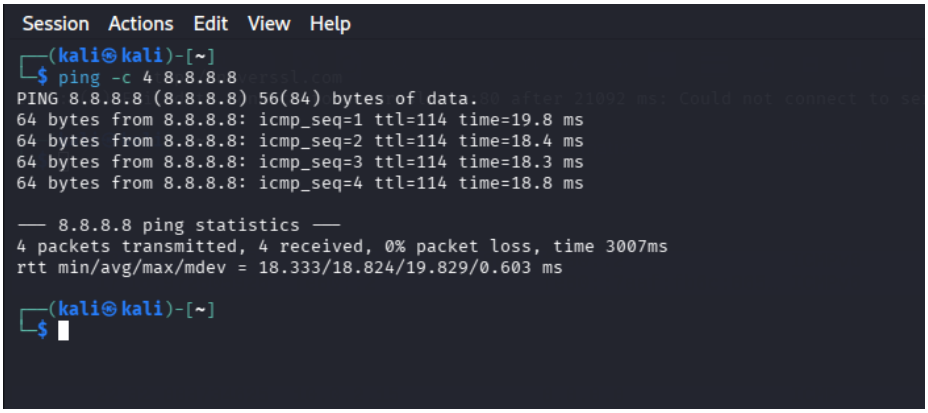


Figure 2: *Terminal output verifying 4 ICMP echo packets transmitted to 8.8.8.8 with 0% packet loss and 18.8ms average RTT*

- **Step 3 (Capture Cessation & File Export):** After 60+ seconds of recording live network frames, stopped packet capture using the red stop button. Exported the raw capture via File > Save As as a standard libpcap file named 'network_traffic.pcap' for offline auditing.

2. Detailed Protocol Analysis & Result Output

Using Wireshark's display filter engine, the captured frames were isolated to inspect protocol behaviors across the Application, Transport, and Network layers.

Protocol 1: Domain Name System (DNS)

DNS operates at the Application Layer (encapsulated over UDP, port 53) to map human-readable domain names to machine-routable IPv4 and IPv6 addresses.

- **Display Filter Used:** 'dns'
- **Packet 5 & 6 (Queries for neverssl.com):** Host 10.0.2.15 transmitted standard queries (Type AAAA and Type A) to local resolver 192.168.100.1 asking for IP addresses corresponding to neverssl.com.
- **Packet 7 & 10 (DNS Responses):** Resolver 192.168.100.1 replied to host 10.0.2.15. Packet 10 returned the IPv4 address 34.223.124.45, completing name resolution.
- **Packet 38 & 39 (Resolution for example.com):** Packet 38 issued a Standard query (Transaction ID 0x4755) for 'A example.com'. Packet 39 returned the server response providing dual IPv4 addresses: 104.20.23.154 and 172.66.147.243.
- **Header Breakdown:** The User Datagram Protocol layer confirms ephemeral client port 33408 communicating with server port 53, requiring zero connection overhead.

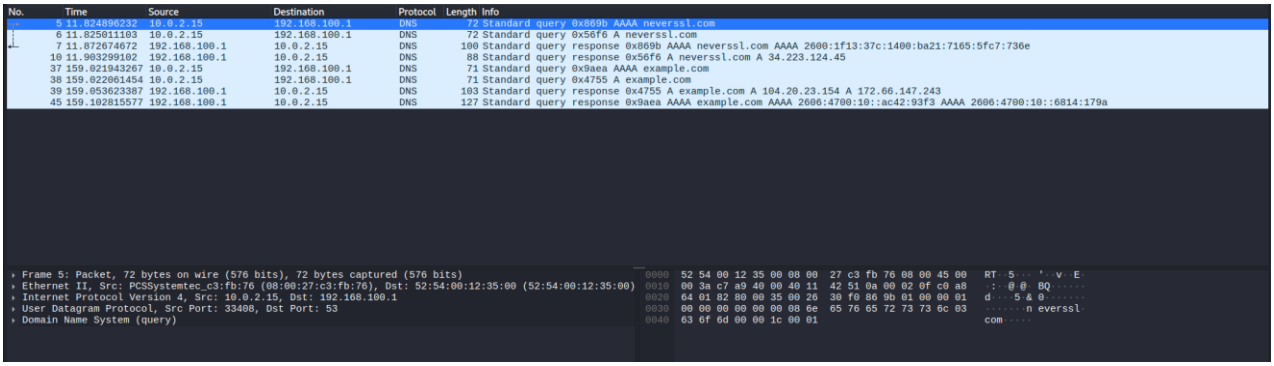


Figure 3: *Filtered Wireshark view for 'dns' demonstrating DNS queries (Packets 5, 6, 38) and successful responses (Packets 7, 10, 39)*

Protocol 2: Internet Control Message Protocol (ICMP)

ICMP operates at the Network Layer (IP protocol 1) to convey operational information, diagnostic queries, and error notifications without transport-layer port assignments.

- **Display Filter Used:** 'icmp'
- **Echo Request Inspection (Packets 22, 25, 27, 29):** Client 10.0.2.15 transmitted four sequential Type 8 Echo Request datagrams to 8.8.8.8 with sequence numbers 1/256 through 4/1024, packet size 98 bytes, and Time-To-Live TTL=64.
- **Echo Reply Inspection (Packets 23, 26, 28, 30):** Target 8.8.8.8 replied with matching Type 0 Echo Reply datagrams (TTL=114). Each reply arrived within 18.3ms–19.8ms, confirming end-to-end layer 3 reachability.

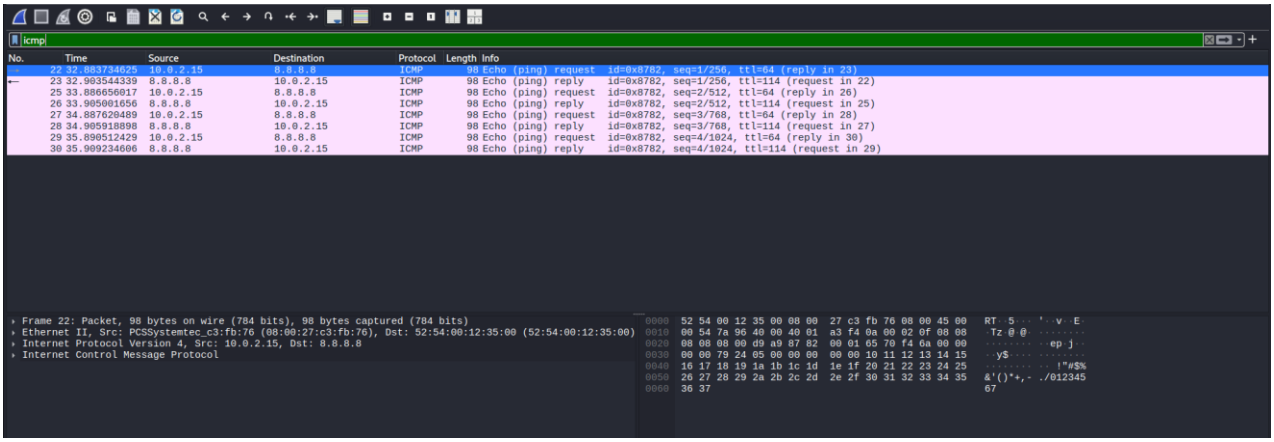


Figure 4: *Filtered Wireshark view for 'icmp' displaying sequential Echo Request and Echo Reply pairs matching terminal ping output*

Protocol 3 & 4: Transmission Control Protocol (TCP) & HTTP

TCP operates at the Transport Layer to establish a reliable, sequenced, and acknowledged connection stream before HTTP application payloads are exchanged.

- **Display Filter Used:** 'tcp'
- **Analysis of TCP Retransmissions (Packets 12–21):** Prior to visiting example.com, an attempt to connect to neverssl.com (34.223.124.45:80) timed out. Wireshark highlighted repeated [TCP

Retransmission] SYN packets in black and red text, demonstrating TCP's automatic retry mechanism when no SYN-ACK is received from a blocked host.

- **TCP Three-Way Handshake (example.com):** When querying example.com (104.20.23.154:80), a healthy 3-way connection was established:
- **1. [SYN] (Packet 40):** Client 10.0.2.15 initiates connection from port 53522 to server port 80 (Seq=0, Win=64240, MSS=1460).
- **2. [SYN, ACK] (Packet 41):** Server responds from port 80 acknowledging connection (Seq=0, Ack=1, Win=65535, MSS=1460).
- **3. [ACK] (Packet 42):** Client confirms handshake completion (Seq=1, Ack=1, Win=64240). Full-duplex connection established.
- **HTTP Application Data Exchange:** Immediately following the handshake, Packet 43 sends 'HEAD / HTTP/1.1' (130 bytes). In Packet 46, the web server responds with 'HTTP/1.1 200 OK' (374 bytes) delivering HTTP headers.
- **Connection Teardown:** Packets 48 through 51 illustrate a graceful four-step connection release using [FIN, ACK] and [ACK] packets.

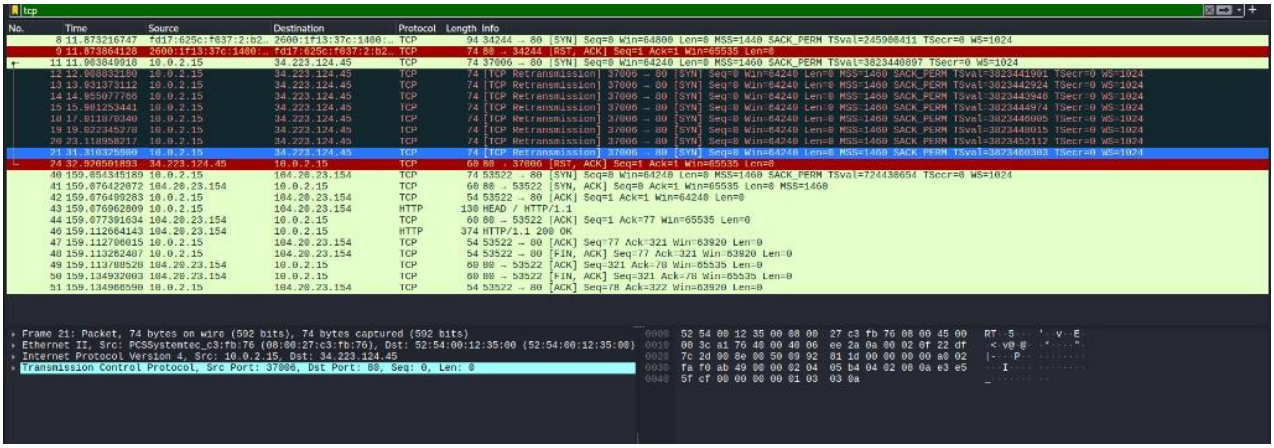


Figure 5: *Filtered Wireshark view for 'tcp' demonstrating connection retries, 3-way handshake, HTTP 200 OK exchange, and FIN teardown*

3. Summary of Technical Findings

- **Protocol Identification:** Successfully identified and analyzed 4 essential network protocols spanning OSI Layers 3 through 7: ICMP (Layer 3), TCP/UDP (Layer 4), and DNS/HTTP (Layer 7).
- **Stateful vs. Stateless Communication:** Contrasted stateless UDP datagrams in DNS and direct ICMP echo queries with stateful TCP handshakes and connection teardowns.
- **Network Diagnostics & Retransmission Behavior:** Observed real-world network troubleshooting phenomena: TCP back-off retransmission timers during destination port unreachable scenarios, alongside clean TCP handshakes and HTTP status code 200 validation.
- **Task Deliverables Completed:** Generated raw packet capture 'network_traffic.pcap' containing all inspected protocol frames, alongside this documentation report.